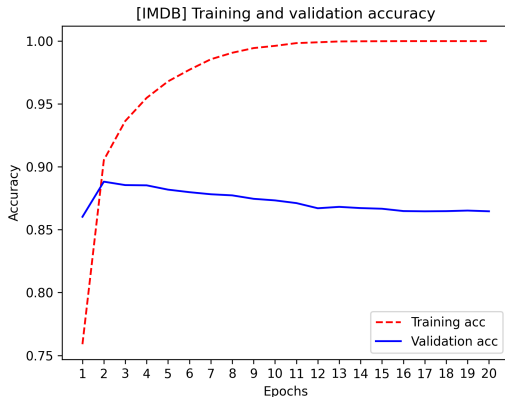


Đánh giá Huấn luyện: Biểu đồ Accuracy

- Training Accuracy (chấm bi) tiến sát 100% (1.0).
- Validation Accuracy (đường liền) đạt đỉnh khoảng 88% tại epoch thứ 4, sau đó giảm dần.
- **Giải pháp:** Ta nên huấn luyện lại một mô hình mới, cài đặt epochs=4 (Early Stopping) để lấy kết quả tốt nhất.



Hình 4.5: Sự biến thiên của Accuracy

Dự đoán dữ liệu mới (Predictions)

Sau khi có mô hình tốt nhất (train 4 epochs), ta có thể sử dụng hàm `predict()` để đánh giá dữ liệu mới ngoài thực tế.

```
1 predictions = model.predict(x_test)
2 print(predictions)
3 # array([[ 0.98006207],
4 #        [ 0.99758697],
5 #        ...,
6 #        [ 0.02885115],
7 #        [ 0.65371346]], dtype=float32)
```

Nhận xét: Mô hình cực kỳ tự tin với một số mẫu (0.99 - rất Tích cực, hoặc 0.02 - rất Tiêu cực). Nhưng với 0.65, nó còn lưỡng lự.

Bài toán Reuters (Multiclass Classification)

- **Mục tiêu:** Phân loại các bản tin tức (Newswires) của hãng thông tấn Reuters vào 46 chủ đề khác nhau.
- **Đặc trưng bài toán:** Đây là bài toán **Single-label, Multiclass** (Mỗi bài báo chỉ thuộc duy nhất 1 chủ đề, nhưng có tới 46 chủ đề để lựa chọn).
- Dữ liệu có 8982 mẫu huấn luyện và 2246 mẫu kiểm tra.
- **Chuẩn bị:** Dữ liệu đầu vào text cũng được dùng Multi-hot Encoding tương tự như IMDb.

Mã hóa Nhãn (One-Hot Encoding Labels)

Ở IMDb có 2 nhãn (0, 1) nên ta dùng số nguyên. Ở Reuters có 46 chủ đề, ta phải biến đổi mảng nhãn thành dạng Vector qua **One-hot encoding** (Mã hóa phân loại).

```
1 from keras.utils import to_categorical
2
3 # Chuyển [3, 10, 45, ...] thành ma trận 46 cột (0 và 1)
4 y_train = to_categorical(train_labels)
5 y_test = to_categorical(test_labels)
6
7 # y_train[0] có nhãn '3' -> sẽ là vector gồm 45 số 0 và số 1 tại index 3.
```

Kiến trúc Mô hình Đa lớp

- **Thất cổ chai thông tin:** Lớp Dense 16 units là quá nhỏ để truyền tải 46 chủ đề. Thông tin sẽ bị mất mát vĩnh viễn. Ta phải tăng lên 64 units.
- **Lớp đầu ra:** Có 46 units (đại diện 46 chủ đề) kết hợp hàm kích hoạt **Softmax**.

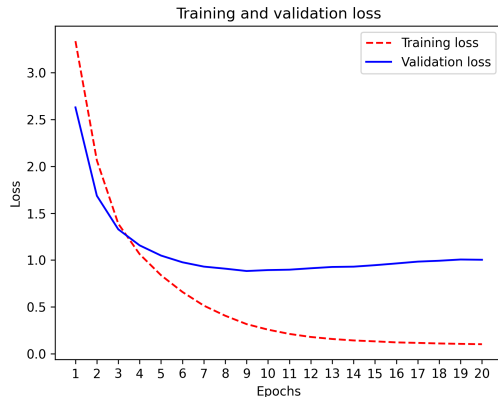
```
1 model = keras.Sequential([  
2     layers.Dense(64, activation="relu"),  
3     layers.Dense(64, activation="relu"),  
4     layers.Dense(46, activation="softmax")  
5 ])
```

Softmax sẽ xuất ra một phân phối xác suất gồm 46 chiều, tổng xác suất luôn bằng 1. Ta cần dùng loss `categorical_crossentropy` để đo khoảng cách giữa 2 phân phối xác suất.

```
1 model.compile(optimizer="adam",  
2               loss="categorical_crossentropy",  
3               metrics=["accuracy"])  
4  
5 history = model.fit(x_train,  
6                   y_train,  
7                   epochs=20,  
8                   batch_size=512,  
9                   validation_split=0.2)
```

Hàm mất mát & Biểu đồ Loss (Reuters)

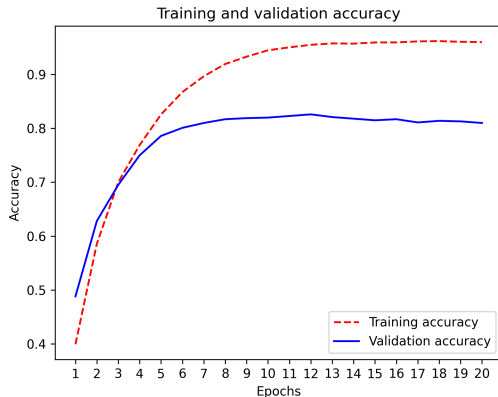
- **Categorical Crossentropy:** Chuyên dụng đo lường khoảng cách giữa hai phân phối xác suất đa chiều.
- Hiện tượng **Overfitting** (Quá khớp) xảy ra ở hầu như mọi bài toán Học sâu. Tại Reuters, Loss trên tập validation bắt đầu tăng mạnh sau epoch thứ 9.



Hình 4.6: Training và Validation Loss (Reuters)

Đánh giá Accuracy

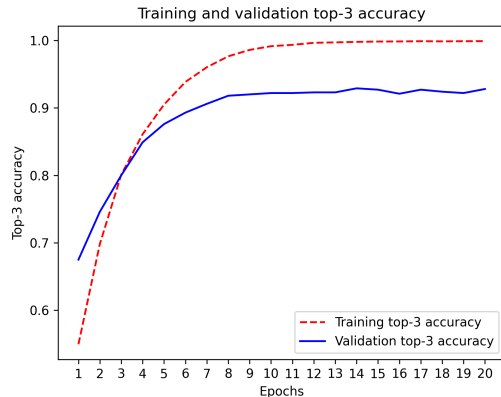
- Accuracy trên tập Validation đạt đỉnh cao nhất khoảng 80% tại epoch thứ 9 trước khi mô hình bị quá khớp.
- So với xác suất chọn ngẫu nhiên (chỉ khoảng 19% nếu đoán bừa theo tần suất tự nhiên của dữ liệu), mức 80% là một kết quả tuyệt vời.



Hình 4.7: Training và Validation Accuracy

Nhưng 80% đã là con số thực? (Top-3 Accuracy)

- Trong tin tức, một bài báo có thể liên quan tới cả 2, 3 chủ đề đan xen. Nếu dự đoán đúng của mô hình là lựa chọn xếp thứ hai hoặc thứ ba thì đó vẫn là thông tin có giá trị.
- Top-3 Accuracy:** Đo lường xem tỷ lệ nhãn thật (Ground-truth) có nằm trong 3 phán đoán có xác suất cao nhất hay không.
- Chỉ số Top-3 này có thể lên tới $>90\%$.



Hình 4.8: Chỉ số Top-3 Accuracy

Bài toán Dự đoán Giá nhà Boston (Scalar Regression)

- **Mục tiêu:** Dự đoán giá nhà trung bình tại vùng ngoại ô Boston (thập niên 1970) dựa trên 13 đặc trưng (tỷ lệ tội phạm, thuế, số phòng...).
- **Khác biệt lớn nhất:** Đầu ra không phải là "xác suất" hay "lớp", mà là **một con số vô hướng liên tục** (Ví dụ: Dự đoán căn nhà giá 25.000 Đô la).
- Tập dữ liệu này cực kỳ nhỏ, chỉ có 506 mẫu. Trong đó 404 mẫu huấn luyện và 102 mẫu kiểm tra.

Tiền xử lý: Chuẩn hóa Dữ liệu (Feature Scaling)

Các đặc trưng dữ liệu (tội phạm, diện tích, tuổi nhà) có thang đo hoàn toàn khác nhau. Đưa trực tiếp vào mạng nơ-ron sẽ gây khó khăn cho hội tụ trọng số. Ta phải chuẩn hóa (**Normalization**): Trừ đi trung bình và chia cho độ lệch chuẩn.

```
1 mean = train_data.mean(axis=0)
2 train_data -= mean
3 std = train_data.std(axis=0)
4 train_data /= std
5
6 # Can phải chuẩn hóa tập test theo trung bình của tập train
7 test_data -= mean
8 test_data /= std
```

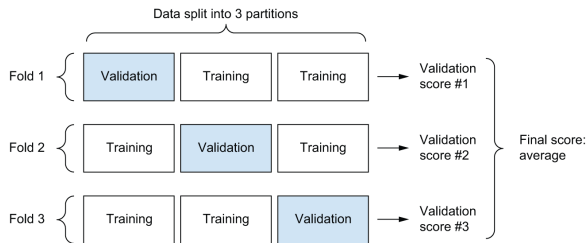
Xây dựng Mạng lưới Hồi quy

Do dữ liệu ít, ta chỉ dùng mạng nơ-ron nhỏ (2 lớp ẩn, 64 units) để tránh Overfitting mạnh.

```
1 def build_model():
2     model = keras.Sequential([
3         layers.Dense(64, activation="relu"),
4         layers.Dense(64, activation="relu"),
5         layers.Dense(1) # Linear Layer - KHÔNG có hàm kích hoạt
6     ])
7     # Loss MSE chuyển để tính lỗi hồi quy
8     # Metric MAE để con người dễ đọc (Mean Absolute Error)
9     model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
10    return model
```

Đặc thù Dữ liệu Nhỏ & K-Fold Cross-Validation

- Tập dữ liệu chỉ có 404 mẫu huấn luyện. Việc tách 100 mẫu ngẫu nhiên ra làm tập Validation sẽ gây ra phương sai sai số rất lớn, phụ thuộc vào "bốc trúng" dữ liệu dễ hay khó.
- **Giải pháp: K-Fold Cross Validation** (Thẩm định chéo K-lần).
- Tách tập train làm K phần bằng nhau ($K = 4$). Mỗi lần dùng 1 phần để validation, 3 phần kia để train. Kết quả là trung bình của K vòng chạy.



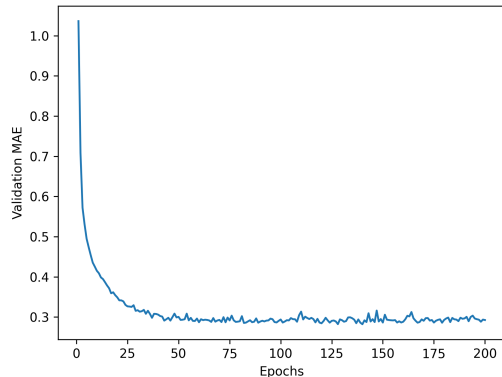
Hình 4.9: Mô phỏng K-Fold Cross-Validation ($K=3$)

Cài đặt K-Fold Validation

```
1 k = 4
2 num_val_samples = len(train_data) // k
3 all_mae_histories = []
4
5 for i in range(k):
6     val_data = train_data[i * num_val_samples: (i + 1) * num_val_samples]
7     val_targets = train_targets[i * num_val_samples: (i + 1) *
8 num_val_samples]
9     # Ghep du lieu cac fold khac de lam tap huan luyen
10    partial_train_data = np.concatenate([
11        train_data[:i * num_val_samples],
12        train_data[(i + 1) * num_val_samples:]], axis=0)
13    ...
14    # model = build_model()
15    # model.fit(...)
```

Đánh giá Hiệu suất Hồi quy (MAE Plot)

- Khác với Accuracy (càng cao càng tốt), MAE (Mean Absolute Error) biểu thị cho sai số trung bình (càng thấp càng tốt).
 $MAE = 0.5$ tương đương sai lệch 500 Đô la.
- Trực quan hóa giá trị trung bình MAE qua K-folds trên 500 epochs.
- Đường đồ thị có độ rung (nhiều) cao ở những epochs đầu.



Hình 4.10: Validation MAE qua 500 epochs